

The three states are:

- **High battery:** The robot is fully charged.
- **Low battery:** The robot is at 15% battery and the warning light is on.
- **Charging:** The robot is docked at the charging point. The three actions are:
 - **Search:** Drive around and clean (high reward, high battery consumption).
 - **Wait:** Stay still and save energy (no reward, no battery use).
 - **Recharge:** Move back to the charging point (no reward, restores battery).

In this example, the robot must decide its actions in the low battery state using the Markov Decision Process. This is where RL comes into action. There are two different scenarios.

- **Risk-take:** The bot chooses to clean more floors (reward, say +10), but there is a 20% probability that the battery will die before it finishes the job. In that case the reward is -50. The probability is known as the Markov Transition.
- **Conservative:** Due to low battery, the bot chooses to recharge. Then it gets 0 reward for cleaning, but it is 100% certain to reach the high battery state for the next cleaning phase.

The logic of reinforcement learning

To handle the two situations the cleaner bot uses a value function. It calculates the expected future reward for both scenarios:

Scenario 1: Expected value: $(0.8 \times 10) + (0.2 \times -50) = 8 - 10 = -2$

Scenario 2: Expected value: $(1.0 \times 0) + (\text{Future High Battery Rewards}) = \text{Positive Value}$

In the first scenario, the expected negative value is worse than a guaranteed path to a full battery. In the second scenario, the RL agent learns that the optimal policy is to head to the charger as soon as the battery is low.

In the real world, the same logic is used for:

- **Mars Rovers:** Deciding whether to explore an unsafe area or stay in the sun to charge solar panels.
- **Drone delivery:** Calculating if a drone has enough battery to reach a customer and return to base.
- **Industrial sensors:** Deciding how often to communicate data to a server to save internal battery life.

Cleaner bot components

Component	Example in a cleaner bot
Agent	The vacuum's software
Environment	The lobby area and charging point
State	Battery level (High/Low)
Action	Search, wait, or recharge
Reward	+10 for cleaning; -50 for a dead battery

A Markov Chain simply describes a sequence of states where the future state depends only on the present one. An MDP adds actions and rewards to that chain.

A Markov Chain-based cleaner bot mechanism decides whether to continue working or return to its charging station based on its battery state.

The concept: From chain to decision

In a standard Markov Chain model, the states and transitions are:

- **States (S):** High battery, low battery.
- **Actions (A):** Clean, wait, recharge.
- **Transition probabilities (P):** The percentage of the battery drops after an action.
- **Rewards (R):** Positive for cleaning, negative for a dead battery.

Here's an implementation in Python.

Define the transition matrices for each action and solve for the best strategy:

```
import numpy as np
# Define States
# 0: High Battery, 1: Low Battery
states = ["High", "Low"]
# Define Rewards for actions in specific states
# Format: [Action_Clean, Action_Wait, Action_Recharge]
rewards = {
    "High": [10, 2, -1], # Best to search when high
    "Low": [-5, 1, 0] # Searching when low risks a
    'dead' battery (-5)
}
# Transition Matrices (The Markov part)
# Rows = Current State, Cols = Next State
# If we Clean
P_search = np.array([
    [0.8, 0.2], # High -> High (80%), High -> Low (20%)
    [0.0, 1.0] # Low -> High (0%), Low -> Low (100% -
    stayed low)
])
# If we 'Recharge'
P_recharge = np.array([
    [1.0, 0.0], # High -> High (100%)
    [1.0, 0.0] # Low -> High (100% - battery recovered)
])
def simulate (current_state_idx, action):
    if action == "search":
        transition_probs = P_search[current_state_idx]
        reward = rewards[states[current_state_idx]][0]
    elif action == "recharge":
        transition_probs = P_recharge[current_state_
idx]
```